



FFR_FD: Effective and fast detection of DeepFakes via feature point defects

Gaojian Wang^{a,b}, Qian Jiang^{a,b}, Xin Jin^{a,b}, Xiaohui Cui^{c,*}

^a Engineering Research Center of Cyberspace, Yunnan University, Kunming, China

^b School of Software, Yunnan University, Kunming, China

^c School of Cyber Science and Engineering, Wuhan University, Wuhan, China

ARTICLE INFO

Article history:

Received 26 August 2021

Received in revised form 14 January 2022

Accepted 3 March 2022

Available online 8 March 2022

Keywords:

DeepFake detection

Feature detection-description

Deep generative models

Face forensics

ABSTRACT

DeepFakes are widespread on social networks, and they result in severe information concerns. Although various detection methods have been proposed, there are still practical limitations. Previous specific artifact-based methods were insufficient to capture fine-grained features, which limited their effectiveness against advanced DeepFakes. Current DNN-based detectors tend to trade high costs for performance improvement, and are not efficient enough, given that DeepFakes can be created easily by mobile apps, and DNN-based models require expensive computational resources. Furthermore, most methods lack generalizability under the cross-dataset scenario. In this work, we instead mine the more subtle and generalized defects of DeepFakes and propose the *fused facial region feature descriptor* (FFR_FD), which is only a vector of the discriminative feature description, for effective and fast DeepFake detection. We show that DeepFake faces have fewer *feature points* than real ones, especially in facial regions. FFR_FD capitalizes on such key observations, and thus has strong generalizability. We train a random forest classifier with FFR_FD to achieve efficient detection. Extensive experiments on six large-scale DeepFake datasets demonstrate the effectiveness of our lightweight method. Our model generalizes well on the challenging Celeb-DF (v2) dataset, with 0.706 AUC, which is superior to most state-of-the-art methods.

© 2022 Elsevier Inc. All rights reserved.

1. Introduction

Misinformation has become ubiquitous [1–3], a consequence of the ability to easily create multimedia content and share it on social platforms. The remarkable progress of deep generative models has further led to the synthesis of super-realistic images or videos. For example, the well-known DeepFake refers to swapping faces in a video through generative adversarial networks (GANs) [4] or autoencoders (AEs) [5,6]. Some mobile apps (e.g., Avatarify¹ and Reface²) also enable facial manipulation. DeepFakes may be abused maliciously [7], such as by creating revenge pornography or striking at politicians. These realistic forgeries can easily deceive the human senses, raising concerns of privacy, social risks, and even national security. Therefore, it is of paramount significance to develop more efficient and generalized methods for DeepFake detection.

* Corresponding author.

E-mail address: xcui@whu.edu.cn (X. Cui).

¹ <https://apps.apple.com/us/app/avatarify-ai-face-animator/id1512669147>

² <https://apps.apple.com/us/app/reface-face-swap-videos/id1488782587>

Early DeepFakes generated by imperfect synthesis algorithms introduced noticeable visual artifacts, inspiring many artifact-based detection methods [9–11] that adapted conventional machine learning models for specific forgery patterns and were computationally efficient. However, with improved means of generating DeepFakes, those artifact patterns no longer hold. Hence, it is necessary to capture more fine-grained discrepancies between real and fake faces, and this is a specialty of deep neural networks (DNNs), whose impressive classification performance has spawned many DNN-based detectors [12–16]. Nevertheless, these models directly take the original image pixels as input, and detection that entirely relies on automatic feature extraction by DNNs is still vulnerable to more advanced DeepFakes [17]. Moreover, most of the above methods are trained for a specific dataset, and cannot extrapolate to other datasets, i.e., they fail to generalize. To improve the effectiveness and generalization of DeepFake detection, much recent work has introduced more powerful modules or domain features to meticulously designed DNNs, such as the attention mechanism [18,19], texture features [20,21], and frequency spectrum [22]. Concomitantly, to drive large and complex DNNs with large-scale datasets requires significant computing resources (e.g., GPUs) and training (e.g., parameter adjustment), which decreases efficiency.

Based on the above observations, DeepFake detection methods struggle to consider effectiveness, efficiency, and generalization. To tackle these issues, we make the following arguments: 1) To improve detection effectiveness, more fine-grained forgery artifacts, which even exist in advanced DeepFakes, should be mined; 2) To improve generalization to unseen datasets, extracted features should be derived from the common defects of DeepFake generative models, and not be specific to any dataset; 3) Considering the increasing incidence of DeepFakes and various forgery technologies, to train an efficient detection model is necessary. Bearing these observations in mind, we proposed to mine more generalizable clues for effective and fast DeepFake detection. From one aspect, it is feasible to capture the subtle artifacts implied in the local features of DeepFakes. Image local feature detection-description algorithms have been successfully used in various vision applications [23,24]. Algorithms to determine feature points by specifying the properties of images (e.g., edges and corners) are called detectors, and those that describe detected points by neighboring pixels are referred to as descriptors. Inspired by the advantages of feature point detector-descriptors, such as fast detection and low-dimensional yet discriminative descriptions, we explore information about the number, distribution, and description of feature points in DeepFake faces.

In essence, deep generative models used for face swaps, such as autoencoders [6], require the reconstruction of faces from latent features. However, the limited encoding space will cause the replaced face to be slightly blurred [13], which will affect the feature point detector that makes decisions through pixel-level differences. Our experiments (Section 4.1) on self-swapped faces and the widely used DeepFake datasets [10,25,16,26,27,17] reveal that the number of feature points in DeepFake faces is generally less than in real ones, especially in the eyes, nose, mouth, and eyebrows. Indeed, due to the concentration of details in these facial regions, the feature points of faces tend to be distributed in them, as shown on the left of Fig. 1(a). Descriptors that record fine-grained features such as gradient information [28] or intensity differences [29,30] will also be affected by blur. However, each feature point of a face has one descriptor, making it arduous to detect DeepFakes through a large number of messy descriptors [31,32]. Based on the above observations, we propose the fused facial region_feature descriptor (FFR_FD), a compact vector representation for DeepFake detection.

FFR_FD can be constructed by various feature point detector-descriptors, and our experiments include SIFT [28], SURF [33], FAST&BRIEF [8,29], ORB [30], and A-KAZE [34]. We divide feature points into eight facial regions: entire face, mouth, inner_mouth, right_eyebrow, left_eyebrow, right_eye, left_eye, and nose. All of the feature point descriptors in each facial region are used to construct a vector FD_r for a region r , which can be controlled to introduce the quantity of information of feature points by not taking the average value during the calculation. We connect FD_r for each facial region in series to form FFR_FD, which is consistent with the computational simplicity of feature detector-descriptors, and significantly reduces the dimension of features extracted from each face, from an $N \times d$ matrix to an $8 \times d$ vector, where N is the total number of feature points in an image, and d is the descriptor's dimension. To subdivide facial regions with FD_r , capitalizes on the lack of feature points of DeepFakes in these regions, and is more discriminative than to directly use all descriptors in the entire face, as shown on the right of Fig. 1(b).

We used FFR_FD as input to train a random forest classifier. Extensive experiments were conducted on six DeepFake benchmark datasets. Results demonstrate the effectiveness of our lightweight model for DeepFake detection. Specifically, the frame-level AUC scores of the challenging DFD [26], DFDC [27], and Celeb-DF_V2 [17] datasets reached 85.2, 88.3, and 82.2, respectively, outperforming many DNN-based detection methods. Moreover, our method has state-of-the-art (SOTA) generalizability, which benefits from the lack of feature points in current DeepFake faces. Frequently used notation in this paper is summarized in Table 1. Our main contributions are as follows:

- Our empirical studies reveal that the use of deep generative models to swap faces will reduce the number of detected feature points;
- The detection and statistical results of many large-scale datasets show that current DeepFake faces have fewer feature points than real faces, especially in detailed regions. This substantial discrepancy is evident even on higher-quality datasets;
- We propose FFR_FD as an informative vector to represent the facial feature description, constructed by feature point descriptors from subdivided facial regions. FFR_FD has computational simplicity and can be combined with various feature point detector-descriptors;
- Random forest trained with FFR_FD can realize SOTA detection. Compared with popular DNN-based detection models, our method is effective, more efficient, and strongly generalizable.

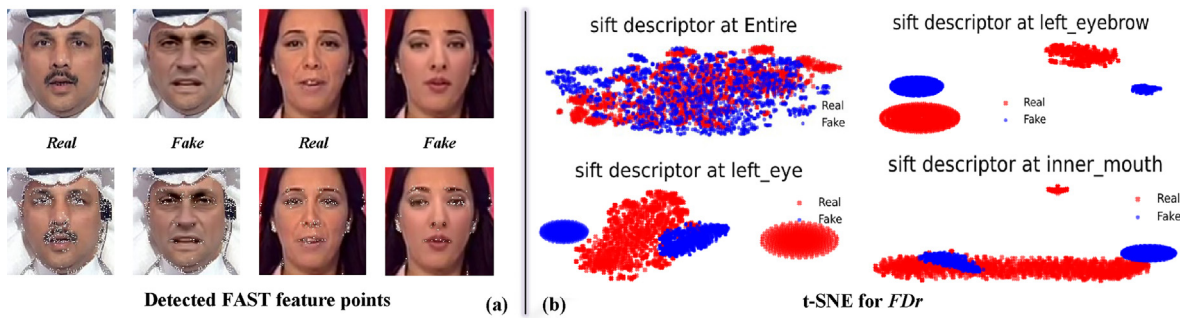


Fig. 1. (a) Two pairs of real and fake images from DeepFakes [16], where white dots are detected FAST [8] feature points. (b) T-SNE visualizations of FD_r in some facial regions, where FD_r is constructed by the SIFT detector-descriptor and comes from 2000 real and 2000 fake images that were randomly selected.

Table 1
Frequently used notation.

Notation	Description
r	Subdivided facial region
FD_r	Proposed feature description for facial region r
FFR_FD	Proposed fused facial region_feature descriptor
N	Number of specific feature points in a face
d	Dimension of a specific feature descriptor

2. Related work

We introduce DeepFake generation and detection methods that are related to our work.

2.1. DeepFake generation and datasets

The rapid progress of deep generative models presents significant challenges to the quick and effective detection of realistic DeepFakes. Many DeepFake video benchmark datasets have been released to promote the research of DeepFake detection. Current DeepFake datasets basically comprise two generations [17]. The first includes UADFV [10], DeepfakeTIMIT [25], and FF++_DeepFakes [16], using FakeApp³, Faceswap-Gan⁴, and Faceswap [6] to generate fake videos. The second generation, including DFD [26], DFDC [27], and Celeb_DF(V2) [17], improves visual quality through upgraded synthesis techniques. The data scale of the second generation has been greatly increased, along with diversity, such as age, skin color, and lighting environment. Due to these advancements, consideration must be given to the cost and speed of a detection model in addition to its performance.

2.2. DeepFake detection

Early work identified visual artifact patterns, such as inconsistent head poses [9], unnatural eye blinking [10], and correlations between facial expressions and head movements [40]. These methods performed well for the time and considered the consumption of the detector by training a simple classifier such as two-class SVM [9,40]. However, those visible artifacts are limited to specific forgeries. Therefore, detection models that adopt deep neural networks (DNNs) have gradually become mainstream. For example, a two-stream CNN was used [12], Meso-4 focused on the mesoscopic properties of images [13], a capsule structure based on VGG19 was used [14], ResNet was used to capture face warping artifacts [15], and classic Xception was used to detect fake faces [16]. With powerful feature extraction capabilities, these DNN-based methods achieved remarkable success, but they still struggle against advanced DeepFakes [17].

Great efforts have been made to apply learning-based detectors to this issue, and most have introduced more domain features or powerful modules. Gram-Net [20] and InTeLe [21] explore the texture information of images to improve robustness. F3-Net [38] uses two frequency-aware branches to extract frequency domain artifacts, with state-of-the-art results in highly compressed settings. The attention mechanism can improve DNN fitting ability, with more focus on critical features, showing great progress in fields such as citywide traffic flow prediction [41] and DeepFake detection. Dang et al. [18] utilized an attention map to locate forged regions and improve detection ability. MADD [19] highlights local parts guided by multi-

³ <https://www.malavida.com/en/soft/fakeapp/>

⁴ <https://github.com/shaoanlu/faceswap-GAN>

Table 2
Comparison of DeepFake detection methods.

Method	Model	Efficient training with consumer CPU	Effective for 2nd-generation DeepFakes	Generalizable to unseen DeepFakes
Handcrafted features or w/o DNN	HeadPose [9]	✓	×	×
	EyeBlinking [15]	×	×	×
	VA-MLP [11]	✓	×	×
	VA-LogReg	✓	×	×
	Meso-4 [13]	✓	×	×
	Shallow CNN	✓	×	×
	Feature Detector/Descriptors [31,32]	×	×	×
	FFR_FD (ours)	✓	✓	✓
DNN-based	Two-stream NN [12]	×	✓	×
	Xception [16]	×	✓	×
	FWA [15]	×	×	×
	DSP-FWA [35]	×	✓	✓
	Multi-task [36]	×	✓	×
	Capsule [14]	×	✓	×
	Face X-ray [37]	×	✓	✓
	F3-Net [38]	×	✓	×
	Two-stream CNN with Xception blocks	×	✓	×
	SMIL [39]	×	✓	×
	MADD [19]	×	✓	✓
	MIL	×	✓	×
	EfficientNet-B4	×	✓	✓
	InceptionV3	×	✓	×
	XceptionNet	×	✓	×
	ResNet-50	×	×	×
	SPPNet	×	✓	✓
	Deep CNN	×	✓	×
	CapsuleNet	×	✓	×
	HRNet	×	✓	✓
	Two-stream CNN with Xception blocks	×	✓	×

attention maps, and combines texture features. Its key motivation is similar to ours, and it emphasizes discrepancies in different facial regions. The above approaches depend on deep networks driven by large-scale data. Currently popular DeepFake detection methods tend to trade high costs for performance improvement, and may still suffer from overfitting. Instead of designing large, complex neural networks or using more modules in DNNs, we efficiently extract discriminative features for effective DeepFake detection.

2.3. Analysis of existing methods

Table 2 compares current methods and the proposed method. Existing DeepFake detection methods can be categorized as follows: 1) Methods without DNN use hand-crafted features for simple binary classifiers. These include some conventional machine learning models [9,11,31,32] and shallow neural networks [13]. Most can operate efficiently on a consumer CPU. Earlier work focused on dataset-specific artifacts, and were effective on DeepFakes of the time. However, they suffered against second-generation DeepFakes, which had significantly fewer obvious artifacts. Furthermore, specific artifacts are difficult to extrapolate to unseen datasets, resulting in limited generalizability; 2) DNN-based methods are being widely explored to improve the effectiveness of detection. Benefiting from the automatic feature extraction capability of DNN, these methods can undoubtedly accomplish second-generation DeepFake detection, since they only need to be retrained and evaluated on the new dataset. Nevertheless, the powerful approximation capacity also limits the performance of most DNN-based methods in cross-dataset evaluation. Recent efforts have noticed overfitting and introduced more general artifacts such as common discrepancies around face-swapping boundaries [37]. In addition to detection performance, it is nontrivial for DNN training to collect large-scale data against emerging forgeries. Models using data-driven DNNs are computationally expensive and cannot be quickly trained using only a consumer CPU.

To meet these practical challenges, we mine the more fine-grained and generalized feature defects of DeepFakes for effective and efficient detection. We reveal that current DeepFake faces have fewer feature points than real ones, and empirically show that this phenomenon stems from deep generative models for face swapping. Our proposed FFR_FD capitalizes on such key observations, which apply to most current DeepFake datasets, and therefore are strongly generalizable. Driven by the distribution of feature points and the discrepancies of descriptors, FFR_FD captures discriminative features in subdivided facial regions for effective DeepFake detection. Moreover, FFR_FD is computationally efficient due to the significantly reduced feature space. Since FFR_FD is a compact vector representation of the face, we use it to quickly train a random forest classifier and achieve efficient detection.

3. Theoretical preliminaries

We discuss various feature point detector-descriptors, which are guided to construct FFR_FD. The detection and description of local features of images is a fundamental problem in computer vision [23,42]. Feature points (also known as interest points or keypoints) are widely used in image matching, retrieval, and recognition tasks. The features extracted by the detec-

tor have different property requirements, depending on the vision task [24]. We are concerned with the effectiveness and efficiency of DeepFake detection.

3.1. SIFT

The scale invariant feature transform (SIFT) is a well-known feature detection and description algorithm [28]. It has four steps: (1) scale-space extrema detection, i.e., the use of difference of Gaussians (DoG) to search for maxima at various scales of the image; (2) keypoint localization, i.e., the location of potential feature points, where the Hessian matrix is used to remove low-contrast points; (3) orientation assignment, i.e., the construction of an orientation histogram based on the gradient information around the feature point, so as to assign an orientation to a feature point; and (4) the keypoint descriptor, i.e., use of the gradient magnitude and orientation to construct a feature vector for feature points. This method extracts a 16×16 neighbor region in each feature point, and subdivides it into 4×4 sub-blocks with eight orientation bins, resulting in a descriptor of $4 \times 4 \times 8 = 128$ dimensions.

3.2. SURF

The feature detection of speeded up robust features (SURF) [33] is faster than that of SIFT. Due to the use of integral images, the boxed filter approximates DOG. SURF uses the determinant of the Hessian matrix to find the interest points at blob-type structures. 2D Haar wavelet responses are used for orientation assignment and feature description. The neighborhood around each feature point is divided into 4×4 subregions, whose wavelet responses have four values, so the descriptor of each feature point is a vector of dimension $4 \times 4 \times 4 = 64$.

3.3. FAST & BRIEF

Features from accelerated segment test (FAST) [8] is a high-speed feature detector developed for real-time applications. Corner point extraction is performed by comparing the intensity thresholds between the point and pixels on a circular ring around it. A decision tree based on the ID3 algorithm is trained as the corner detector. Non-maximum suppression is used to solve the problem of adjacent corner points, which makes the FAST detector more efficient. FAST is only a feature point detector and does not involve feature description. Binary robust independent elementary features (BRIEF) [29] is a feature point descriptor that uses simple binary strings. It performs Gaussian smoothing for the patch partition of detected feature points, and randomly selects pairs of points for intensity difference tests to obtain descriptors. If the number of pairs of points is 256, then the dimension of the descriptor is 32 bytes for each feature point, which is much smaller than that of SIFT or SURF. BRIEF performs similarly to SIFT in most scenarios, but its low complexity makes the construction of descriptors faster. Combining the FAST detector and BRIEF descriptor has the advantages of fast speed and low computational resource requirements.

3.4. ORB

Introduced by Rublee et al. [30], oriented FAST and rotated BRIEF (ORB) solves the problem that the FAST detector does not compute orientation and the BRIEF descriptor lacks rotational invariance. ORB applies the Harris corner measure to pick the Top N points from FAST corners to evaluate cornerness and determine the local orientation by the intensity centroid [30]. ORB extracts the BRIEF descriptor according to the main orientation of the feature point. It adopts a greedy algorithm to find random pairwise pixel patches with low correlation. In this study, the authors selected 256 pairwise pixel patches for feature description, so it generates a 32-dimensional descriptor. Note that we use the feature point information for DeepFake detection. The extracted faces have been cropped and aligned, so the improved properties of ORB have little impact on detection, but we still use it for comparison.

3.5. A-KAZE

Introduced by Alcantarilla et al. [34], the A-KAZE feature detector and descriptor effectively constructs the nonlinear scale space for feature detection through FED [43]. The descriptor is based on M-LDB, which is also efficient at exploring gradient information from the nonlinear scale space.

4. Investigating feature points of DeepFakes and real faces

We explore the number and distribution of feature points between DeepFake and real faces. Our results show that using deep generative models to swap the face will reduce detectable feature points, especially in detailed regions. Statistics on six challenging DeepFake datasets further demonstrate our findings.

4.1. Swapping faces to reduce the number of feature points

Will deep generative models cause a change in the number of facial feature points? To this end, we utilize the *original faceswap model* [6] for face swapping. The model is based on an autoencoder-decoder: the shared encoder is trained for two faces, A and B, to extract the latent feature vectors, from which the corresponding faces are reconstructed by their respective decoders. The compressed latent vectors can be understood as general face information such as the expression and background, and the decoders are used to restore specific facial features, such as nose details, from latent vectors. After training, the latent vectors extracted from A by the shared encoder are transmitted to the decoder of B, such that A is replaced by B, as shown in Fig. 2.

Because the feature points of diverse facial appearances are naturally different, we use one face as both A and B, i.e., we swap a face with itself. A real-face video containing 396 frames from FF++_YouTube [16] was used for experiments. After sufficient training of the model, we swapped the face with itself and detected the SIFT, SURF, FAST, ORB, and A-KAZE feature points. The feature points of an original face and the corresponding fake face are shown in Fig. 3. Obviously, the fake face has fewer feature points than the real face with the same feature point detector. This phenomenon is more noticeable in facial regions with detailed information, such as the nose, mouth, eyes, and eyebrows. It can also be seen from Fig. 3 that while the swapped face is similar to the original, many refined details for determining feature points are lost.

To verify the observation results, we counted the numbers of feature points on the original and swapped faces from the corresponding 369 video frames. The feature points in the facial images extracted from frames were summed, and the total was divided by the number of faces to obtain the average number of feature points for each face. Based on the discrepancies of the feature point distribution, we used Dlib [44] to subdivide the face into seven regions: mouth, inner_mouth, right_eyebrow, left_eyebrow, right_eye, left_eye, and nose. We calculated the average number of feature points, as shown in Table 3. These results again show that the Faceswap model will reduce the numbers of feature points of faces, especially in the subdivided facial regions. Compared to other detectors, the number of feature points of fake faces is reduced more significantly on FAST and ORB (based on modified FAST). Both visualizations and statistical results inspire us to explore the feature points on detailed facial regions to detect forged faces.

4.2. Average number of feature points on DeepFake datasets

Does the defect of fewer feature points exist in a DeepFake meticulously crafted with various advanced generation and optimization technologies? We further observed the distribution of feature points on faces from DeepFake datasets, and similarly divided the facial region to calculate the average number of feature points of real and fake faces. We used all images

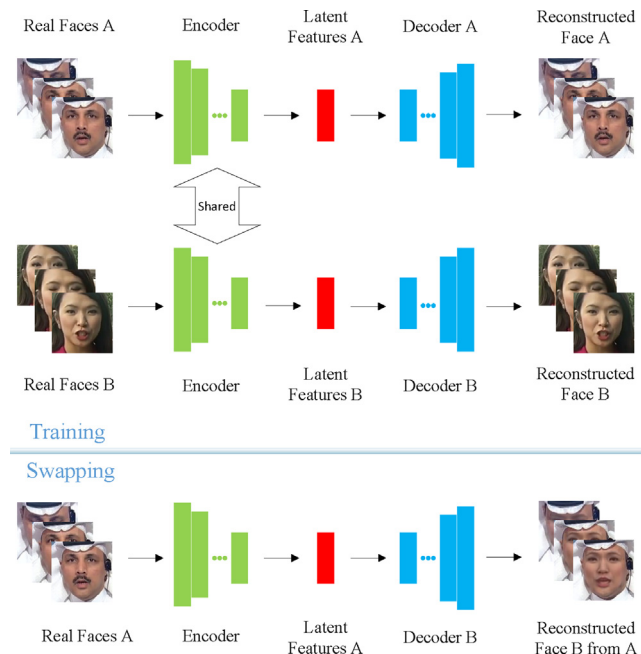


Fig. 2. Typical procedures to create a DeepFake based on an autoencoder-decoder. The swapped face is generated from the limited encoding space.

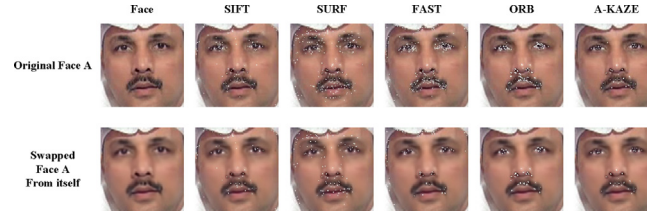


Fig. 3. Five types of feature points detected in original and self-swapped faces. DeepFake lacks corresponding feature points, especially in facial regions such as the eyes, nose, and mouth.

from the training sets, and present the statistics in Table 4. The results show that both the first- and second-generation DeepFake datasets lack sufficient feature points. This motivates us to explore the use of feature point information for detection.

5. Methodology

It is infeasible to directly judge authenticity by the number of feature points because this fluctuates according to factors such as facial appearance and video quality (compare FF++_DF (RAW) and FF++_DF (LQ) in Table 4). Hence, it is necessary to explore the combination of feature point descriptors for detection, which is nontrivial if there are many feature points. If N feature points are detected in a facial image, and the descriptor of each feature point is d -dimensional (depending on the descriptor, as shown in Table 5), then one-shot will produce $(N \times d)$ -dimensional features. In current large-scale DeepFake datasets, to directly train the classifier with descriptors [31,32] or BOWs [45] (constructed from descriptors by k-means) consumes much time and is prone to overfitting. We propose FFR_FD for fast, effective DeepFake detection.

5.1. FFR_FD

Algorithm 1: Construction of FFR_FD

Input: A facial image I

Output: FFR_FD (or FFR_FD_ave)

```

1: Set  $DET$  to the specified feature-point detector
2: Set  $DES$  to the specified feature-point descriptor
3: Set  $ROI$  with [mouth, inner-mouth, right-eyebrow, left-eyebrow, right-eye, left-eye, nose]
4: for  $r$  in  $ROI$  do
5:    $FR[r] = facial\_landmarks(r)$ 
6:    $KP = DET(I)$ 
7:   for  $r$  in  $FR$  do
8:      $N[r] = 0$ 
9:      $FD[r] = \mathbf{0}$ 
10:    for  $p$  in  $KP$  do
11:      if  $p \in r$  then
12:         $FD[r] = FD[r] + DES(p)$ 
13:         $N[r] = N[r] + 1$ 
14:      else
15:        continue
16:     $FD[r]_{ave} = FD[r] / N[r]$ 
17: for  $r$  in  $FR$  do
18:    $FFR\_FD = concatenate(FD[r])$ 
19:    $FFR\_FD\_ave = concatenate(FD[r]_{ave})$ 
20: return  $FFR\_FD$  (or  $FFR\_FD\_ave$ )

```

The construction of FFR_FD is described in Algorithm 1, and illustrated in Fig. 4. We first run a face detector and extract 68 facial landmarks for each face with DLib [44]. According to these, we divide the face into eight rectangular regions (r)—entire face, mouth (49–68), inner_mouth (62–68), right_eyebrow (18–22), left_eyebrow (23–27), right_eye (37–42), left_eye (43–48), and nose (28–36)—where the lack of feature points is further magnified, and their descriptors from the same region have certain similarities. We detect feature points in the facial image, and assign the calculated descriptors to the corresponding

regions. Assume that N_r feature points are detected in region r , and the descriptor of feature point DES is d -dimensional. We add the descriptors of the N_r feature points according to the dimensions to obtain the feature descriptor of local facial region r ,

$$FD_r[i] = \sum_{N_r} DES[i], \quad i = 0, 1, \dots, d-1. \quad (1)$$

FD_r is a vector that reduces the dimensionality of the feature from $N_r \times d$ to $1 \times d$. Note that if we do not average FD_r (i.e., we do not divide by N_r), information about the number of feature points is introduced. FD_r accumulates the gradient information (for SIFT and A-KAZE), intensity difference (for FAST&BRIEF and ORB), or wavelet response (for SURF) of all feature point descriptors in the specified facial region. If no feature points are detected in a region, then FD_r of that region is populated with a d -dimensional vector of 0. Finally, we concatenate FD_r of the eight regions in sequence to obtain an $(8 \times d)$ -dimensional vector, FFR_FD.

The time complexity of Algorithm 1 strongly depends on the face detector, feature point detector-descriptor, and number of feature points. If l is the time of face detection, m is the time of detecting all feature points in a facial image, and n is the time of generating descriptors for each feature point, then the complexity is $O(l + m + (n+1)N)$. The algorithm obtains facial landmarks through the face detector and subdivides the facial regions, with complexity $O(l)$. Then the complexity of detecting all N feature points in the face and computing the corresponding descriptor is $O(m + N * n)$. After that, N feature points are traversed and divided into facial regions to construct FD_r , which is $O(N)$, and the concatenation of the FD_r to obtain FFR_FD is constant.

FFR_FD has the following characteristics: 1) It integrates the feature descriptions of all subdivided facial regions; 2) Its calculations are efficient. FFR_FD can be constructed by combining any feature point detector and descriptor, so its distinguishability and speed benefit from the feature point algorithm; 3) It can control whether it is affected by the number of feature points; 4) Only a low-dimensional vector ($8 \times d$) is used to describe the facial feature information, which significantly reduces the dimension compared to the original facial image (256×256) or feature descriptors ($N \times d$), as shown in Table 5.

5.2. Differences in FFR_FD between real and fake faces

If the number of real images in a training set is N_R , then N_R FFR_FDs are generated. We calculate their means and variances on the $8 \times d$ dimensions, and similarly, N_F for fake images. In a given dimension, we subtract the statistical results of fake images from those of real ones to obtain the statistical differences in each dimension, as shown in Fig. 5 (without averaging FD_r). In UADFV, the mean values of FFR_FDs constructed by the SIFT detector-descriptor for real faces are larger than those of fake faces in all dimensions, especially in the entire face and mouth regions, as shown in Fig. 5(a). Note that SIFT descriptors are based on gradients; hence, the gradients around the feature points of real faces are stronger than those of fake faces. Fig. 5(b) shows that the real FFR_FDs have a greater variance in the entire face, i.e., their fluctuation is more significant. Feature points detected by FAST are described by BRIEF. In Fig. 5(c), in the mouth, inner_mouth, right_eye, and left_eye regions, the mean values of the real faces are larger, i.e., the intensity differences used to construct the BRIEF descriptor are greater, and the opposite is true in the right_eyebrow, left_eyebrow, and nose regions. Fig. 5(d) illustrates that the real faces have greater intensity difference fluctuations, especially in the mouth and inner_mouth. Similar analyses can be performed on other datasets, such as Celeb-DF_V2, as shown at the bottom of Fig. 5. We conclude that real facial images and DeepFakes are fairly different in FFR_FD, and the specific distribution is affected by the feature point detector-descriptor.

5.3. DeepFake detection based on FFR_FD

We introduce a procedure for efficient DeepFake detection based on FFR_FD: 1) The DeepFakes video dataset is split into training, validation, and testing sets, and all videos are extracted into frames, at rates set proportionally to balance the number of real and fake frames; 2) We extract and align faces from frames with the S3Fd Detector and Fan Aligner [6]. The size of each face is 256×256 ; 3) As discussed in Section 5.1, we construct FFR_FD as a vector representation from each original face;

Table 3

Average number of feature points for eight facial regions between original and corresponding self-swapped faces.

FP Detector → Region ↓ Face →	SIFT		SURF		FAST		ORB		A-KAZE	
	ori	swap	ori	swap	ori	swap	ori	swap	ori	swap
entire face	126.7	91.6	182.6	141.1	146.1	53.7	237.2	108.6	67.5	50.7
mouth	23.9	10.1	43.3	29.1	46.4	8.7	70.2	17.9	23.8	14.5
inner_mouth	23.9	3.6	18.8	13.0	21.8	2.9	32.7	17.9	9.4	3.3
right_eyebrow	4.2	3.0	18.8	9.8	3.5	0.4	1.2	0.1	0.6	0.3
left_eyebrow	4.2	2.2	13.2	8.0	7.6	2.3	6.1	3.3	1.6	1.1
right_eye	10.4	4.4	8.5	6.5	21.1	7.8	37.5	15.4	8.8	7.4
left_eye	11.0	6.7	8.8	7.4	20.0	10.4	39.8	21.0	8.7	7.8
nose	11.0	10.4	24.0	18.8	23.1	12.3	55.3	33.7	8.7	11.6

Table 4

Average number of feature points for faces from DeepFake datasets. Based on five feature detector-descriptors, current DeepFakes struggle to provide enough feature points across facial regions.

Detector→		SIFT		SURF		FAST		ORB		A-KAZE		Detector →		SIFT		SURF		FAST		ORB		A-KAZE	
Dataset ↓	Region ↓	real	fake	real	fake	real	fake	real	fake	real	fake	Dataset ↓	Region ↓	real	fake	real	fake	real	fake	real	fake	real	fake
DT(HQ)	entire face	77.9	68.9	173.9	159.7	99.2	65.7	107.1	88.1	33.9	34.3	FF++_DF(LQ)	entire face	94.6	81.1	181.7	169.3	123.7	91.6	161.5	98.8	48.4	37.5
	mouth	8.0	5.2	22.5	19.5	13.6	6.6	15.8	9.9	5.7	5.0		mouth	9.0	4.9	19.3	14.8	20.3	8.6	31.3	13.0	8.9	5.3
	inner_mouth	4.4	2.8	7.8	6.6	8.5	4.0	11.4	7.2	3.6	3.0		inner_mouth	9.0	2.7	7.2	5.2	12.7	5.3	22.6	9.2	5.8	3.4
	right_eyebrow	1.4	1.3	5.9	5.2	2.5	1.5	1.0	0.7	0.9	0.9		right_eyebrow	2.9	2.4	7.8	6.9	6.6	5.4	5.9	4.3	3.0	2.5
	left_eyebrow	1.5	1.4	6.3	5.7	3.0	2.1	1.0	1.1	0.8	1.2		left_eyebrow	2.9	2.4	7.9	6.7	6.4	5.2	5.5	4.4	2.9	2.4
	right_eye	4.1	3.2	5.5	4.9	9.9	6.4	19.6	14.5	0.8	3.8		right_eye	5.5	2.9	5.2	4.2	11.0	6.1	24.6	13.0	5.4	3.9
	left_eye	4.4	3.2	5.3	5.0	10.5	6.6	21.1	16.0	4.2	4.2		left_eye	5.4	2.9	5.3	4.3	10.9	6.0	25.0	13.2	5.5	4.0
DT(LQ)	nose	8.0	7.7	15.8	15.6	13.4	9.6	28.7	29.8	11.4	12.4		nose	7.6	5.6	14.8	12.3	10.5	5.5	26.8	15.7	9.3	7.2
	entire face	77.9	57.9	173.9	130.2	99.2	38.6	107.1	45.2	33.9	21.1	DFD	entire face	118.7	100.2	195.9	183.5	193.8	131.4	208.2	160.0	52.7	50.0
	mouth	8.0	3.5	22.5	15.2	13.6	2.1	15.8	3.4	5.7	2.3		mouth	15.1	7.8	26.3	23.0	32.1	13.4	46.9	25.4	11.6	9.7
	inner_mouth	4.4	1.9	7.8	4.6	8.5	1.6	11.4	2.8	3.6	1.6		inner_mouth	9.9	4.4	10.7	9.5	20.7	13.4	35.9	17.7	7.9	6.3
	right_eyebrow	1.4	0.8	5.9	3.0	2.5	0.2	1.0	0.1	0.9	0.1		right_eyebrow	2.6	1.9	6.8	6.0	7.7	4.5	4.7	3.5	1.7	1.5
	left_eyebrow	1.5	0.8	6.3	3.6	3.0	0.3	1.0	0.2	0.8	0.3		left_eyebrow	2.5	2.0	7.6	6.3	8.3	4.7	4.5	3.2	1.5	1.3
	right_eye	4.1	2.2	5.5	4.5	9.9	3.1	19.6	7.2	0.8	2.4		right_eye	6.1	4.3	5.4	5.0	13.7	8.4	24.2	18.0	4.6	4.6
	left_eye	4.4	2.2	5.3	4.6	10.5	3.5	21.1	8.2	4.2	2.6		left_eye	6.1	4.5	5.6	5.2	14.3	8.9	25.0	19.3	4.7	4.9
UADFV	nose	8.0	5.9	15.8	11.7	13.4	3.4	28.7	15.1	11.4	9.1		nose	10.8	9.1	19.6	17.0	14.3	15.1	46.5	38.7	14.0	13.6
	entire face	96.4	74.8	186.1	160.6	108.2	116.7	148.3	71.9	40.8	22.3	DFDC	entire face	83.0	70.0	152.8	131.7	99.4	78.8	134.1	94.9	41.5	28.2
	mouth	9.3	3.0	20.0	11.8	16.2	14.1	27.2	9.3	8.1	2.5		mouth	9.0	7.0	18.7	16.5	16.1	12.4	25.8	18.8	7.7	5.2
	inner_mouth	5.6	2.1	7.1	4.6	9.7	8.4	18.8	6.4	5.3	1.8		inner_mouth	6.1	4.8	8.6	7.7	11.6	9.4	20.8	15.7	5.4	3.9
	right_eyebrow	2.4	2.7	8.6	4.6	4.4	9.8	3.4	1.7	2.0	1.5		right_eyebrow	2.1	1.7	7.1	6.4	3.3	2.9	3.2	2.3	1.4	0.9
	left_eyebrow	2.3	2.4	7.9	8.2	4.7	9.9	3.8	2.3	2.0	1.4		left_eyebrow	2.2	1.6	8.3	6.4	4.2	2.4	3.3	1.5	2.1	0.6
	right_eye	6.1	3.3	4.8	4.4	11.4	10.5	23.8	10.9	4.7	2.9		right_eye	4.9	4.2	5.4	5.0	6.8	6.1	13.4	11.7	3.4	2.7
	left_eye	5.8	3.0	4.8	4.6	11.2	10.3	24.4	11.9	4.6	2.7		left_eye	4.9	4.1	5.9	5.3	7.2	6.0	14.9	11.7	3.4	2.6
FF++_DF(RAW)	nose	10.1	6.0	19.5	18.9	14.8	15.5	36.4	14.9	11.8	6.0		nose	9.5	7.8	17.8	16.7	14.0	12.3	34.6	27.3	10.7	9.0
	entire face	122.4	100.3	209.4	190.5	158.8	98.2	215.1	128.8	57.5	44.5	Celeb-DF_V2	entire face	58.0	46.2	140.1	121.3	54.8	31.7	87.2	46.8	34.3	22.7
	mouth	12.7	5.3	22.6	16.6	26.2	6.4	42.6	15.1	10.9	6.3		mouth	7.2	3.6	18.8	14.9	10.3	4.0	18.3	6.9	6.8	3.4
	inner_mouth	8.0	3.0	8.3	5.9	16.4	4.4	31.7	11.0	7.3	4.1		inner_mouth	7.2	1.9	7.5	5.1	7.2	2.6	14.1	4.8	6.8	2.4
	right_eyebrow	3.1	2.4	8.7	7.3	7.8	4.6	6.8	4.1	3.4	2.6		right_eyebrow	1.5	0.9	5.6	3.9	2.0	0.6	1.8	0.4	1.4	0.7
	left_eyebrow	3.1	2.4	9.0	7.2	7.7	4.5	6.4	4.1	3.3	2.5		left_eyebrow	1.5	0.9	5.8	3.9	2.0	0.6	1.7	0.4	1.4	0.6
	right_eye	8.2	4.9	6.3	5.6	15.0	8.0	34.7	20.9	6.8	5.4		right_eye	3.3	2.2	4.4	3.9	5.8	3.4	12.2	6.6	3.6	2.6
	left_eye	8.1	4.8	6.5	5.6	14.9	7.8	35.1	21.2	7.0	5.5		left_eye	3.3	2.3	4.5	4.1	5.7	3.5	12.6	7.2	3.7	2.8
	nose	10.7	7.5	18.9	15.3	16.6	7.1	39.8	24.0	11.3	9.1		nose	7.1	5.9	16.6	14.6	7.9	4.6	23.0	14.5	9.6	7.6

Table 5
Dimensions of feature point descriptors and FFR_FD.

Algorithm	Descriptor Dimensions	FFR_FD Dimensions
SIFT	128	1024 (8×128)
SURF	64	512 (8×64)
FAST&BRIEF	32	256 (8×32)
ORB	32	256 (8×32)
A-KAZE	61	488 (8×61)

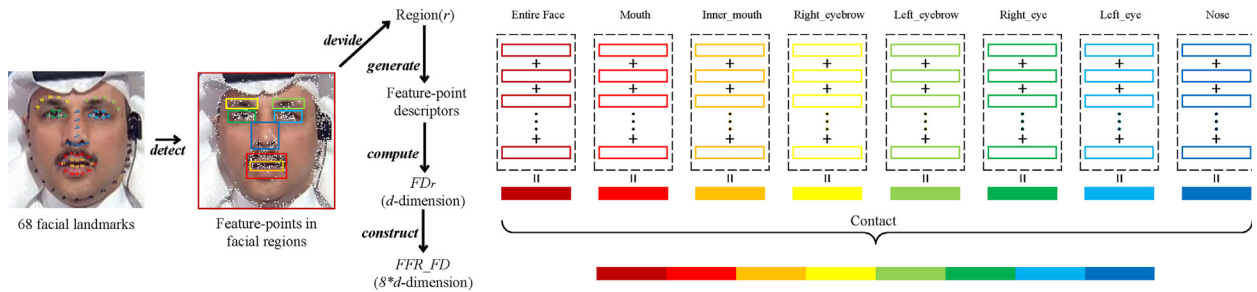


Fig. 4. Construction of FFR_FD. Hollow rectangles represent feature point descriptors. Colors correspond to facial regions.

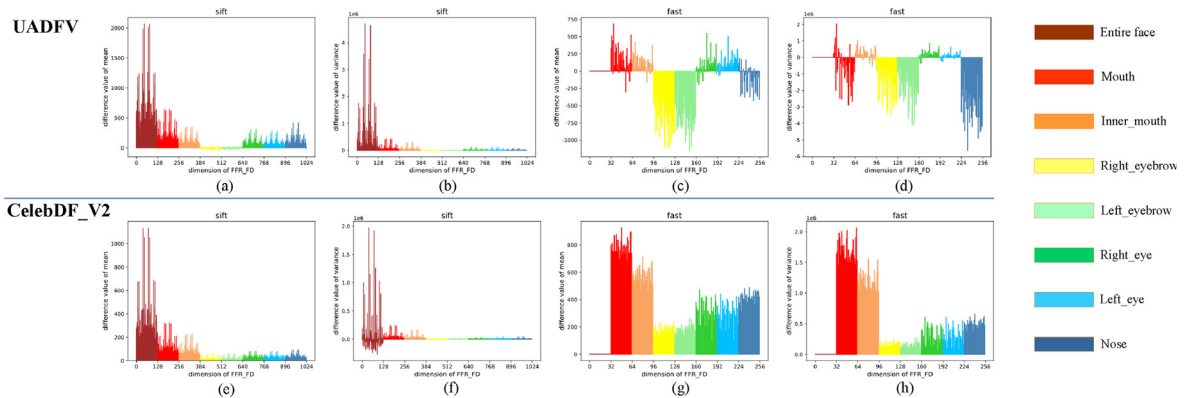


Fig. 5. Differences of FFR_FD between real and fake faces. Results are differences between statistical indicators in each dimension of FFR_FD, including the mean and variance, where the difference value is used as the ordinate. The feature point algorithm used to construct FFR_FD is marked above each graph. Images are from the UADFV (top) and Celeb-DF_V2 (bottom) training sets.

4) We only use FFR_FD as inputs to learn the random forest classifier for the binary classification task, i.e., real or fake, as shown in Fig. 6.

We tried other models, such as LR, SVM, ELM, and MLP, and found that random forest performs best in our experiments, as it does not easily overfit. Random forest works well with high-dimensional features, and has a strong adaptability to features and a fast training speed, which is compatible with the characteristics of FFR_FD. Also, random forest can measure the importance of features, which has some reference significance. Hence, the random forest classifier presents many advantages for our proposed FFR_FD in DeepFake detection:

- It can handle high-dimensional data without handcrafted feature selection, which accords with the feature description of FFR_FD;
- It is relatively robust to noise, which is helpful given that some FFR_FDs are constructed from poor-quality DeepFake faces;
- It provides feature importance in classification, which explains crucial facial regions for detection;
- It is computationally efficient on large datasets due to the parallelization of subtrees in training, which is consistent with the motivation of FFR_FD for fast detection.

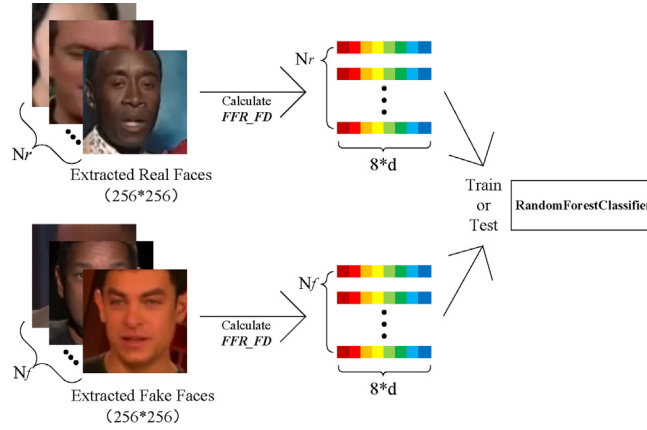


Fig. 6. Simple pipeline of proposed FFR_FD for DeepFake detection.

Algorithm 2: Algorithm for training random forest classifier with FFR_FD

Input: Training set: $D = \{(x_i, y_i) | i = 1, 2, \dots, m\}$, where x_i is the $FFR_FD \in \mathbf{R}^{8 \times d}$ and y_i is the corresponding label;
 The number of CART trees in the forest: T

Output: Random forest classifier for DeepFake detection: *Model*

1: **for** $t = 1$ **to** T **do**

2: Draw the bootstrap set D_t of size m from D .

3: Randomly select $\sqrt{8 \times d}$ features from the original $(8 \times d)$ -dimensional features as F (max features)

4: Grow a decision tree DT_t from D_t :

5: **while** $F \neq \emptyset$ **and** $D_n \geq \min_{\text{samples}}$ **do**

6: Compute the impurity (loss) function with the *Gini index*, which acts as the criterion for candidate split $\theta(f, \alpha_{\min})$:

$$\text{Gini} = 1 - \sum_{y_i} p_{y_i}^2$$

7: For data D_n with N_n samples at node n , select the optimal feature f among F to split D_n into D_n^{left} and D_n^{right} :

$$L(D_n, \theta) = \frac{D_n^{\text{left}}}{N_n} \text{Gini}(D_n^{\text{left}}(\theta)) + \frac{D_n^{\text{right}}}{N_n} \text{Gini}(D_n^{\text{right}}(\theta))$$

8: Choose parameters of θ that minimize the impurity function $L(D_n, \theta)$: $\theta^* \leftarrow \arg\min_{\theta} (L(D_n, \theta))$

9: Recurse for next: $F \leftarrow F \setminus \{f\}$, $D_n \leftarrow D_n^{\text{left}}(D_n^{\text{right}})$

10: Get the ensemble of decision trees as random forest classifier (*Model*): $\{DT_t\}_1^T$

11: **return** *Model* trained by FFR_FD .

Algorithm 2 describes the training of a random forest classifier with FFR_FD, which in our case consists of CART decision trees, each providing a vote, i.e., the class probability between 0 (real) and 1 (fake). The *Gini criterion* guides the construction of the decision tree to select the optimal attribute from the $8 \times d$ -dimensional features of FFR_FD as a split node. After training, the model predicts the testing data through voting. In our model, as deployed by the open-source scikit-learn [47], this is the average of the predicted probabilities from T decision trees,

$$P(\text{Model}) = \frac{1}{T} \sum_{t=1}^T P(DT_t). \quad (2)$$

6. Experiments and results

We describe our experimental setup, and demonstrate the effectiveness of the proposed method through extensive experiments.

6.1. Experimental setup

Datasets

We evaluated six widely-used first- and second-generation DeepFake datasets in our experiments. The first-generation included DeepfakeTIMIT (HQ and LQ) [25], UADFV [10], and FF++_DeepFake (RAW and LQ) [16], and the second-generation datasets were DFD [26], DFDC [27], and Celeb-DF_V2 [17]. Each dataset was split into 80% for training and the rest for testing. For Celeb-DF_V2, we followed the officially specified label [17] to divide the testing set. The facial datasets for our experiment are shown in Table 6; to prevent data leakage, facial images were extracted after the original videos were split into training and testing sets. DeepFakeTIMIT and UADFV provide fewer frames, so all were used. In all other datasets, we only used a small part of the frames in each video. The data scale was reduced to take advantage of the efficiency of FFR_FD and evaluate whether our method required as much data as deep CNN.

Implementation

FFR_FD We used the feature detector-descriptors of SIFT, SURF, FAST&BRIEF, ORB, and A-KAZE to construct FFR_FD for comparison. To investigate the influence of quantity information of feature points for detection, we compared the detection results of FFR_FD obtained both by averaging and not averaging FD_r , which are denoted as *ave* and *no_ave*, respectively.

Hyperparameter settings We report the frame-level ROC-AUC score as the metric, which is widely used in DeepFake detection. In our experiments, 20% of the training data were sequentially split as the validation subset for fivefold cross-validation. The two most crucial parameter definitions in the random forest classifier are the number of decision subtrees T and max feature F (i.e., the number of features for each node to grow a subtree). Breiman [46] pointed out that increasing the number of subtrees will not lead to overtraining, since the generalization error always converges. As shown in Table 5, the feature dimension of the proposed FFR_FD is not very large, so we set the range of T only to include 200, 500, 800, and adjusted the number of decision trees according to the optimal metrics reported on the validation subsets. As for the max feature F , although it can be increased to improve the strength of each tree in the model, its reduction will also benefit from the reduced correlation between trees in terms of the ensemble. We developed the random forest classifier with scikit-learn [47], which provides the *auto* mode in the *RandomForest* class to automatically select the optimal F . Under this setting, m was fixed at $\sqrt{8xd}$, where $8xd$ is the dimension of the proposed FFR_FD. Other hyperparameters followed the scikit-learn default. Our code is available at Github here.

An Nvidia GeForce RTX 2080 Ti GPU was used to extract the faces, and other experiments were performed on a consumer-level Intel Core i7-9700 K CPU with 32 GB RAM.

6.2. Comparison with previous methods

We evaluate our method on six DeepFake datasets and compare FFR_FD with previous detection methods, as shown in Table 7. Among five detector-descriptors, the overall detection performance of FFR_FD extracted by the FAST detector combined with the BRIEF descriptor was most effective and stable. As described in Section 4, the number of feature points of fake faces detected by FAST was significantly less. However, ORB with the same significant reduction in the number of feature points did not perform ideally. ORB is based on FAST&BRIEF to improve scale- and rotation-invariance, but all faces were cropped and aligned in the DeepFake detection task. In this case, scale- and rotation-invariance weaken the discriminability of FFR_FD. As shown in Table 7, the introduction of feature point quantity information did not improve AUC in some cases, indicating that the feature point descriptors themselves are sufficiently distinguishable. Comparing the detection results of the RAW and c40 (LQ) versions of FF++_DF, our method was affected slightly by strong compression, but was still acceptable. On the DT (LQ), UADFV, and extremely challenging DFDC and Celeb-DF_V2 datasets, our detection performance was ahead of all methods, and it was competitive on other datasets.

DSP-FWA [15] and Xception [16] achieved better AUCs than our method on DT (HQ) and FF++_DF, respectively. FWA [15] identified artifacts introduced by affine face warping, and used a CNN (Res50) to detect DeepFakes. Based on FWA, DSP-FWA further introduced a spatial pyramid pooling module to solve multiscale issues. This method of exposing face warping artifacts was especially effective for frontal facial images, and DSP-FWA achieved a higher AUC score on the face-centered DT

Table 6
Details of datasets for experiments.

Dataset\Num→	Released frames		Training set		Testing Set	
	Real	Fake	Real	Fake	Real	Fake
DeepfakeTIMIT (HQ)	34.0 k	34.0 k	27.0 k	27.0 k	7.0 k	7.0 k
DeepfakeTIMIT (LQ)	34.0 k	34.0 k	27.0 k	27.0 k	7.0 k	7.0 k
UADFV	17.3 k	17.3 k	8.2 k	8.2 k	9.1 k	9.1 k
FF++_DeepFakes (raw)	509.9 k	509.9 k	29.0 k	29.0 k	9.6 k	9.6 k
FF++_DeepFakes (c40)	509.9 k	509.9 k	13.4 k	13.4 k	3.1 k	3.1 k
DFD	315.4 k	2,242.7 k	30.3 k	30.0 k	10.0 k	10.0 k
DFDC	488.4 k	1,783.3 k	19.0 k	19.0 k	4.5 k	4.8 k
Celeb-DF_V2	225.4 k	2,116.8 k	38.0 k	38.0 k	10.0 k	10.0 k

Table 7

AUC (%) scores of our method and other detection methods on DeepFake datasets. The best results among all methods and between FFR_FDs are highlighted with **bold** and red, respectively.

AUC (%) Methods ↓	Datasets → FFR_FD types ↓		DT (HQ)	DT (LQ)	UADFV	FF++_DF		DFD	DFDC	Celeb-DF_V2
						(raw)	(c40)			
Ours	SIFT	ave	60.8	97.3	96.9	84.4	79.2	81.2	81.5	81.6
		no_ave	60.5	97.1	92.4	84.4	79.3	84.3	80.5	81.7
	SURF	ave	44.9	86.1	97.1	78.9	74.9	60.4	88.3	82.2
		no_ave	63.0	84.8	92.8	79.5	75.2	66.0	84.6	81.5
	FAST&BRIEF	ave	84.0	99.6	96.5	89.6	81.1	77.3	80.0	80.5
		no_ave	85.1	99.9	93.2	92.3	81.6	85.2	69.9	78.0
	ORB	ave	50.7	98.6	90.5	81.5	77.5	69.8	78.6	76.1
		no_ave	65.4	98.1	87.4	84.9	77.6	76.3	67.8	74.1
	A-KAZE	ave	41.8	78.1	97.8	81.1	77.5	56.3	82.0	79.9
		no_ave	48.2	86.7	96.3	79.2	74.0	62.2	76.6	79.5
Two-stream NN [12]			73.5	83.5	85.1	70.1	-	52.8	61.4	53.8
Meso-4 [13]			68.4	87.8	84.3	84.7	-	76.0	75.3	54.8
MesoInception-4			62.7	80.4	82.1	83.0	-	75.9	73.2	53.6
HeadPose [9]			53.2	55.1	89.0	47.3	-	56.1	55.9	54.6
FWA [15]			93.2	99.9	97.4	80.1	-	74.3	72.7	53.9
VA_mLP [11]			62.1	61.4	70.2	66.4	-	69.1	61.9	55.0
VA-LogReg			77.3	77.0	54.0	78.0	-	77.2	66.2	55.1
Xception-raw [16]			54.0	56.7	80.4	99.7	-	53.9	49.9	48.2
Xception-c40			70.5	75.8	83.6	95.6	-	65.8	69.7	65.5
Multi-task [36]			65.8	62.2	55.3	76.3	-	54.1	53.6	54.3
Capsule [14]			74.4	78.4	61.3	96.6	-	64.0	53.3	57.5
DSP-FWA [35]			99.7	99.9	97.7	93.0	-	81.1	75.5	64.6

(HQ) dataset. However, on the more challenging second-generation DeepFake datasets, its detection performance decreased significantly because the improved DeepFakes reduced obvious artifacts and provided more angles of faces. The proposed FFR_FD captured the discrepancies in fine-grained feature descriptions and was only limited to detectable facial regions. Xception is a powerful deep CNN, and it produced SOTA results on FF++_DF. Note that it is driven by a total of nearly 193 K frames from FF++_DF videos [16], which is much more than ours, and its better performance was expected.

It is worth mentioning the following advantages of our method: 1) FFR_FD is just a vector of length $8 \times d$ (shown in Table 5), and the length is only 256 for one face if constructed based on FAST&BRIEF; 2) Driven by large-scale training data or features, models such as two-stream (a two-stream CNN), FWA/DSP-FWA (ResNet-50), Xception, multi-task (CNN-based), and capsule (VGG19-based) require expensive GPU resources to train large and complex CNNs. We only used a consumer-level CPU to train the random forest, with comparable or even better results; 3) Methods that also use simple machine learning models to capture specific features, such as HeadPose (SVM) and VA (multilayer perceptron or logistic regression), are far from satisfactory because captured superficial visual artifacts are not refined enough, resulting in limited detection capabilities, and they are easy to repair. However, deep generative models are challenged to generate DeepFakes with sufficient feature points; DeepFakes lacked fine-grained information (e.g., gradient, intense differences) for feature point detection. This makes our approach SOTA; 4) FFR_FD benefits from feature point detector-descriptors such as speed and reliability, and FAST&BRIEF is the first choice. In summary, our method has advantages in speed, performance, storage, and computational costs.

6.3. Generalization ability evaluation

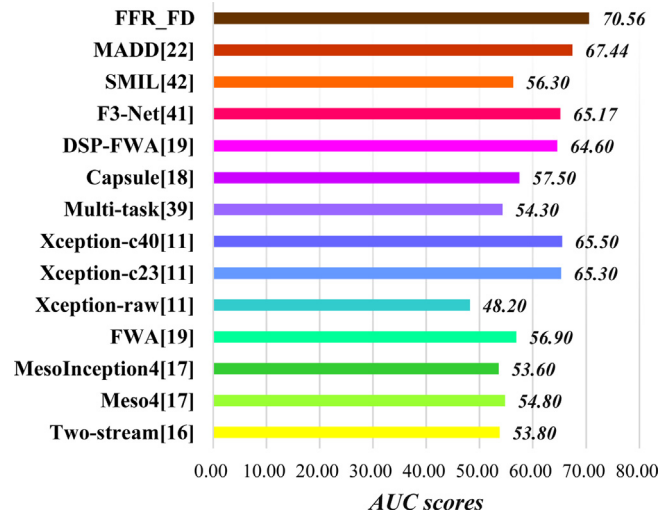
Although DNN-based models showed excellent DeepFake detection performance, many are dataset-specific and cannot extrapolate to unseen datasets. To evaluate the generalization ability of the proposed FFR_FD, we conducted cross-dataset experiments, in which the classifier was trained on one dataset and tested on others. In this setting, we only used the random forest with 200 subtrees trained by DeepfakeTIMIT (HQ) as the classifier, and all the other datasets were used as testing sets to evaluate generalization. The task is nontrivial because DeepfakeTIMIT (HQ) is a first-generation DeepFake dataset with few frames available. Other datasets had more DeepFake videos, of better quality. We believe we are the first to test generalization in this challenging setting. ResNet50 [48], Xception [49], and EfficientNet-B0 [50] are SOTA DNN-based DeepFake detectors, so we chose them as baselines. We implemented baseline models with *Keras* and *TensorFlow* libraries, and only modified the fully-connected layers for binary classification.

The generalization results of baselines and FFR_FD (constructed by FAST&BRIEF) are shown in Table 8. ResNet50, Xception, and EfficientNet-B0 achieved perfect AUC scores on DT (LQ), slightly better than 98.5% of the result of our FFR_FD. Indeed, these deep CNNs effectively extract discriminative features, but they are limited to in-domain data. Comparing

Table 8

AUC (%) scores on other datasets for the classifier only trained with DeepFakeTIMIT (HQ).

Methods		Train	Test						
			DT (LQ)	UADFV	FF++_DF(RAW)	FF++_DF(LQ)	DFD	DFDC	Celeb-DF_V2
FAST&BRIEF	ave	DT (HQ)	25.8	67.7	50.0	48.6	52.3	44.6	39.9
	no_ave		98.5	67.7	80.0	73.1	67.8	52.8	68.8
ResNet50			100.0	48.5	49.4	53.2	53.2	53.4	51.3
Xception			100.0	50.3	50.0	50.0	56.4	51.6	50.2
EfficientNetB0			100.0	50.0	50.0	50.0	54.1	54.1	53.2

**Fig. 7.** AUC(%) score of cross-dataset evaluation on Celeb-DF_V2.

ave and *no_ave*, the introduction of feature point quantity information can improve generalizability, benefiting from the lack of feature points in most DeepFake datasets. FFR_FD constructed only from DeepfakeTIMIT (HQ) was also generalizable to cross-domain datasets, except DFDC. Behind this result, DFDC involves multiple synthesis algorithms, and FFR_FD constructed from DFDC has little similarity with DeepFakeTIMIT. Results show that our method has considerable generalization ability, while baseline methods are limited to the training set and suffer from overfitting.

6.4. Comparison with other SOTA methods on Celeb-DF_V2

The Celeb-DF_V2 dataset improves the forgery quality and significantly reduces visible artifacts compared to other datasets, and hence is more challenging to generalize. Following convention [17,19], we trained the model on FF++ and tested it on Celeb-DF_V2 (i.e., *no_ave* FFR_FD constructed from FF++ was used to train the model, and *no_ave* FFR_FD from Celeb-DF_V2 was used to test it), to evaluate the transferability of our method. The frame-level average AUC score is shown in Fig. 7. Compared with the recently proposed MADD [19], F3-Net [38], and SMIL [39], FFR_FD further improved the metrics by 3.12%, 6.27%, and 14.26%, respectively. The results demonstrate that our method achieves SOTA generalizability. Behind this improvement on the challenging Celeb-DF_V2, the facial regions of forged faces lack sufficient feature points, although the synthesis algorithms are enhanced. Moreover, the *feature importances* of FFR_FD focused by random forest actually show a similar distribution on FF++ and Celeb-DF_V2, as shown in Fig. 8(d) and (h). These provide new insights into the resulting gains of generalization performance.

6.5. Feature importance

We divide the face into eight regions, comprising the entire face, mouth, inner_mouth, right_eyebrow, left_eyebrow, right_eye, left_eye, and nose, to construct FFR_FD. Are these facial regions informative for DeepFake detection? To answer this question, we output the feature (Gini) importance of random forest. This is the normalized total reduction of the criterion brought by a feature. Fig. 8 visualizes the feature importance of FFR_FD constructed by FAST&BRIEF on different datasets. On DeepfakeTIMIT, FF++_DeepFakes, DFD, and Celeb-DF_V2, the mouth region is the crucial feature for identification. As shown in Fig. 9, the forged faces in these datasets have artifact defects on the mouth, which causes the feature point descriptors of the mouth to struggle with the real ones. Fig. 8 also shows that most facial regions contribute to the random forest

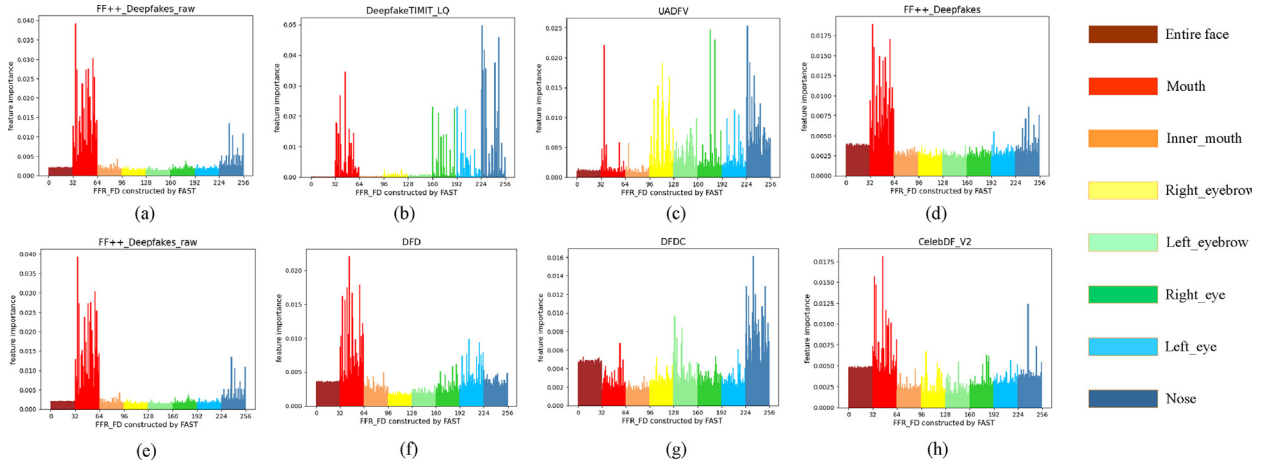


Fig. 8. Feature importance of FFR_FD for random forest classification. Results come from FFR_FD constructed by FAST detector and BRIEF descriptor. Dataset is marked above each subgraph.



Fig. 9. Real and corresponding DeepFake faces; DeepFakes struggle with the details of the mouth region.

classification with FFR_FD. Hence, when detecting DeepFakes in the wild, we should not use prior knowledge of defects in some regions of a specific dataset.

7. Efficiency of FFR_FD

In our experiments, both the construction of FFR_FD and the training of random forest were performed on a consumer-level Intel Core i7-9700 K CPU with 32 GB RAM. Fig. 10(a) shows the average calculation time to construct an FFR_FD from a 256×256 facial image. The quantitative comparison illustrates that the computational efficiency of the feature detection-descriptor for computing FFR_FD is ranked as $FAST \& BRIEF > ORB > A-KAZE > SURF > SIFT$. As expected, it depends on the calculation speed of the feature point algorithm [24].

We randomly used 10 K FFR_FDs (i.e., extracted from 10 K images) to train a random forest classifier with 500 subtrees, and compared average times of the training procedure. As shown in Fig. 10(b), the training time is positively related to the dimension of FFR_FD. In our experimental environment, if 100 K facial images are used and FFR_FD is constructed from FAST&BRIEF, it takes less than one hour to complete the construction and training process. Mainstream DNN-based models are competitive in DeepFake detection tasks. However, even with the help of a GPU, driving 100 K facial images to train deep neural networks consumes more time than our method.

8. Limitations and conclusion

We presented FFR_FD, a vector representation for DeepFake detection, which can be constructed from different facial regions in combination with various feature descriptors. Inspired by local feature detection-description algorithms to extract fine-grained features, we explored the feature points in DeepFakes. Using feature point detector-descriptors SIFT, SURF, FAS-

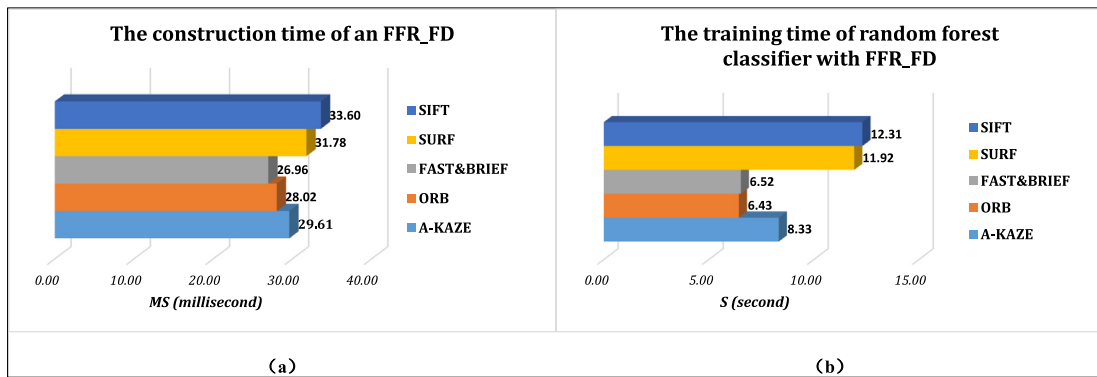


Fig. 10. (a) Average construction time of FFR_FD. (b) Average training time of random forest classifier with 10 K FFR_FDs.

T&BRIEF, ORB, and A-KAZE, experimental results indicate that current DeepFake faces lack a sufficient number of feature points. Without the use of GPUs, a random forest classifier with the proposed FFR_FD can achieve state-of-the-art detection performance while considering efficiency and generalization.

FFR_FD relies heavily on feature point detector-descriptors, but current algorithms are not specifically designed for DeepFake detection tasks, given that they must compromise between distinguishability and invariance. In future work, we would like to design a more discriminative feature descriptor for face forensics.

CRedit authorship contribution statement

Gaojian Wang: Conceptualization, Methodology, Software, Visualization, Investigation, Writing - original draft. **Qian Jiang:** Funding acquisition, Formal analysis, Writing - review & editing, Data curation. **Xin Jin:** Software, Validation, Visualization, Resources. **Xiaohui Cui:** Writing - review & editing, Project administration, Supervision, Resources.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research was funded by the National Natural Science Foundation of China (No. 62002313), Key Areas Research Program of Yunnan Province in China (202001BB050076), Key Laboratory in Software Engineering of Yunnan Province in China (2020SE408), and Postdoctoral Science Foundation of Yunnan Province in China.

Appendix A. Normalization of FFR_FD

In our experiments, FFR_FD does not need to be normalized, given that feature point descriptors have been normalized. SIFT, BRIEF, ORB, and A-KAZE are all normalized at $[0, 255]$, and the SURF descriptor based on the sum of the Haar wavelet response is at $[-1, 1]$. We construct FD_r by accumulating the same type descriptors in the facial region r , and the resulting FFR_FDs have dimensional unification. Compared with logistic regression, support vector machine, and multilayer perceptron, random forest is the best classifier according to our test results. It does not need to standardize the features, given that the decision tree does not involve distance measurement.

References

- [1] D. Caputo, L. Verderame, A. Ranieri, A. Merlo, L. Caviglione, Fine-hearing google home: why silence will not protect your privacy, *J. Wireless Mobile Networks, Ubiquitous Computing Dependable Appl. (JoWUA)* 11 (1) (2020) 35–53.
- [2] D.J. Lemay, R.B. Basnet, T. Doleck, Examining the relationship between threat and coping appraisal in phishing detection among college students, *J. Internet Services Inform. Secur. (JISIS)* 11 (1) (2020) 38–49.
- [3] V. Sihag, M. Vardhan, P. Singh, G. Choudhary, S. Son, De-lady: Deep learning based android malware detection using dynamic features, *J. Internet Services Inform. Secur. (JISIS)* 11 (2) (2021) 34–45.
- [4] I.J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, Y. Bengio, Generative adversarial networks, *arXiv preprint arXiv:1406.2661* (2014).
- [5] D.P. Kingma, M. Welling, Auto-encoding variational bayes, *arXiv preprint arXiv:1312.6114* (2013).
- [6] faceswap github, url: <https://github.com/deepfakes/faceswap>. Accessed:2021-03.
- [7] A.O. Kwok, S.G. Koh, Deepfake: a social construction of technology perspective, *Current Issues Tourism* 24 (13) (2021) 1798–1802.

- [8] E. Rosten, T. Drummond, Machine learning for high-speed corner detection, in: *European conference on computer vision*, Springer, 2006, pp. 430–443..
- [9] X. Yang, Y. Li, S. Lyu, Exposing deep fakes using inconsistent head poses, in: *ICASSP 2019–2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, IEEE, 2019, pp. 8261–8265.
- [10] Y. Li, M.-C. Chang, S. Lyu, In ictu oculi: Exposing ai created fake videos by detecting eye blinking, in: *2018 IEEE International Workshop on Information Forensics and Security (WIFS)*, IEEE, 2018, pp. 1–7..
- [11] F. Matern, C. Riess, M. Stamminger, Exploiting visual artifacts to expose deepfakes and face manipulations, in: *2019 IEEE Winter Applications of Computer Vision Workshops (WACVW)*, IEEE, 2019, pp. 83–92..
- [12] P. Zhou, X. Han, V.I. Morariu, L.S. Davis, Two-stream neural networks for tampered face detection, in: *2017 IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, IEEE, 2017, pp. 1831–1839.
- [13] D. Afchar, V. Nozick, J. Yamagishi, I. Echizen, Mesonet: a compact facial video forgery detection network, in: *2018 IEEE International Workshop on Information Forensics and Security (WIFS)*, IEEE, 2018, pp. 1–7..
- [14] H.H. Nguyen, J. Yamagishi, I. Echizen, Use of a capsule network to detect fake images and videos, *arXiv preprint arXiv:1910.12467* (2019)..
- [15] Y. Li, X. Yang, P. Sun, H. Qi, S. Lyu, Celeb-df: A large-scale challenging dataset for deepfake forensics, *arXiv preprint arXiv:1811.00656* (2018)..
- [16] A. Rossler, D. Cozzolino, L. Verdoliva, C. Riess, J. Thies, M. Nießner, Faceforensics++: Learning to detect manipulated facial images, in: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019, pp. 1–11.
- [17] Y. Li, X. Yang, P. Sun, H. Qi, S. Lyu, Celeb-df: A large-scale challenging dataset for deepfake forensics, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 3207–3216.
- [18] H. Dang, F. Liu, J. Stehouwer, X. Liu, A.K. Jain, On the detection of digital face manipulation, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 5781–5790.
- [19] H. Zhao, W. Zhou, D. Chen, T. Wei, W. Zhang, N. Yu, Multi-attentional deepfake detection, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 2185–2194.
- [20] Z. Liu, X. Qi, P.H. Torr, Global texture enhancement for fake face detection in the wild, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 8060–8069.
- [21] X. Sun, B. Wu, W. Chen, Identifying invariant texture violation for robust deepfake detection, *arXiv preprint arXiv:2012.10580* (2020)..
- [22] X. Zhang, S. Karaman, S.-F. Chang, Detecting and simulating artifacts in gan fake images, in: *2019 IEEE International Workshop on Information Forensics and Security (WIFS)*, IEEE, 2019, pp. 1–6..
- [23] A. Mian, M. Bennamoun, R. Owens, On the repeatability and quality of keypoints for local feature-based 3d object retrieval from cluttered scenes, *Int. J. Comput. Vision* 89 (2) (2010) 348–361.
- [24] S.A.K. Tareen, Z. Saleem, A comparative analysis of sift, surf, kaze, akaze, orb, and brisk, in: *2018 International conference on computing, mathematics and engineering technologies (iCOMET)*, IEEE, 2018, pp. 1–10..
- [25] P. Korshunov, S. Marcel, Deepfakes: a new threat to face recognition? assessment and detection, *arXiv preprint arXiv:1812.08685* (2018).
- [26] N. Dufour, A. Gully, Contributing data to deepfake detection research, *Google AI Blog* (2019).
- [27] B. Dolhansky, R. Howes, B. Pfau, N. Baram, C.C. Ferrer, The deepfake detection challenge (dfdc), 2019, preview dataset, *arXiv preprint arXiv:1910.08854*.
- [28] D.G. Lowe, Distinctive image features from scale-invariant keypoints, *Int. J. Comput. Vision* 60 (2) (2004) 91–110.
- [29] M. Calonder, V. Lepetit, C. Strecha, P. Fua, Brief: Binary robust independent elementary features, in: *European conference on computer vision*, Springer, 2010, pp. 778–792..
- [30] E. Rublee, V. Rabaud, K. Konolige, G. Bradski, Orb: An efficient alternative to sift or surf, in: *2011 International conference on computer vision*, IEEE, 2011, pp. 2564–2571..
- [31] F.F. Kharbat, T. Elams, A. Mahmoud, R. Abdullah, Image feature detectors for deepfake video detection, in: *2019 IEEE/ACS 16th International Conference on Computer Systems and Applications (AICCSA)*, 2019, pp. 1–4.
- [32] Z. Akhtar, D. Dasgupta, A comparative evaluation of local feature descriptors for deepfakes detection, in: *2019 IEEE International Symposium on Technologies for Homeland Security (HST)*, IEEE, 2019, pp. 1–5.
- [33] H. Bay, A. Ess, T. Tuytelaars, L. Van Gool, Speeded-up robust features (surf), *Comput. Vision Image Understanding* 110 (3) (2008) 346–359.
- [34] P.F. Alcantarilla, T. Solutions, Fast explicit diffusion for accelerated features in nonlinear scale spaces, *IEEE Trans. Patt. Anal. Mach. Intell.* 34 (7) (2011) 1281–1298.
- [35] K. He, X. Zhang, S. Ren, J. Sun, Spatial pyramid pooling in deep convolutional networks for visual recognition, *IEEE Trans. Pattern Anal. Mach. Intell.* 37 (9) (2015) 1904–1916.
- [36] H.H. Nguyen, F. Fang, J. Yamagishi, I. Echizen, Multi-task learning for detecting and segmenting manipulated facial images and videos, in: *2019 IEEE 10th International Conference on Biometrics Theory, Applications and Systems (BTAS)*, IEEE, 2019, pp. 1–8.
- [37] L. Li, J. Bao, T. Zhang, H. Yang, D. Chen, F. Wen, B. Guo, Face x-ray for more general face forgery detection, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 5001–5010.
- [38] Y. Qian, G. Yin, L. Sheng, Z. Chen, J. Shao, Thinking in frequency: Face forgery detection by mining frequency-aware clues, in: *European Conference on Computer Vision*, Springer (2020) 86–103.
- [39] X. Li, Y. Lang, Y. Chen, X. Mao, Y. He, S. Wang, H. Xue, Q. Lu, Sharp multiple instance learning for deepfake video detection, in: *Proceedings of the 28th ACM international conference on multimedia*, 2020, pp. 1864–1872.
- [40] S. Agarwal, H. Farid, Y. Gu, M. He, K. Nagano, H. Li, Protecting world leaders against deep fakes., in: *CVPR Workshops* (2019) 38–45.
- [41] A. Ali, Y. Zhu, M. Zakarya, Exploiting dynamic spatio-temporal correlations for citywide traffic flow prediction using attention based neural networks, *Inf. Sci.* 577 (2021) 852–870.
- [42] N. Wang, X. Gao, D. Tao, H. Yang, X. Li, Facial feature point detection: A comprehensive survey, *Neurocomputing* 275 (2018) 50–65.
- [43] S. Grewenig, J. Weickert, A. Bruhn, From box filtering to fast explicit diffusion, in: *Joint Pattern Recognition Symposium*, Springer, 2010, pp. 533–542..
- [44] D.E. King, Dlib-ml: A machine learning toolkit, *J. Mach. Learn. Res.* 10 (2009) 1755–1758.
- [45] Y. Zhang, L. Zheng, V.L. Thing, Automated face swapping and its detection, in: *2017 IEEE 2nd International Conference on Signal and Image Processing (ICSIP)*, 2017, pp. 15–19.
- [46] L. Breiman, Random forests, *Mach. Learn.* 45 (1) (2001) 5–32.
- [47] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, et al., Scikit-learn: Machine learning in python, *J. Mach. Learn. Res.* 12 (2011) 2825–2830..
- [48] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [49] F. Chollet, Xception: Deep learning with depthwise separable convolutions, in: *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 1251–1258.
- [50] M. Tan, Q. Le, Efficientnet: Rethinking model scaling for convolutional neural networks, in: *International Conference on Machine Learning*, PMLR, 2019, pp. 6105–6114..